



至此，你掌握了编写组织有序而易于使用的程序所需的基本技能，该考虑让程序目标更明确、用途更大了。在本章中，你将学习处理文件，让程序能够快速分析大量的数据；你将学习错误处理，避免程序在面对意外情形时崩溃；你将学习**异常**，它们是Python创建的特殊对象，用于管理程序运行时出现的错误；你还将学习模块json，它让你能够保存用户数据，以免在程序停止运行后丢失。

学习处理文件和保存数据可让你的程序使用起来更容易：用户将能够选择输入什么样的数据，以及在什么时候输入；用户使用你的程序做一些工作后，可将程序关闭，以后再接着往下做。学习处理异常可帮助你应对文件不存在的情形，以及处理其他可能导致程序崩溃的问题。这让你的程序在面对错误的数时更健壮——不管这些错误数据源自无意的错误，还是源自破坏程序的恶意企图。你在本章学习的技能可提高程序的适用性、可用性和稳定性。

10.1 从文件中读取数据

文本文件可存储的数据量多得难以置信：天气数据、交通数据、社会经济数据、文学作品等。每当需要分析或修改存储在文件中的信息时，读取文件都很有用，对数据分析应用程序来说尤其如此。例如，你可以编写一个这样的程序：读取一个文本文件的内容，重新设置这些数据的格式并将其写入文件，让浏览器能够显示这些内容。

要使用文本文件中的信息，首先需要将信息读取到内存中。为此，你可以一次性读取文件的全部内容，也可以以每次一行的方式逐步读取。

10.1.1 读取整个文件

要读取文件，需要一个包含几行文本的文件。下面首先来创建一个文件，它包含精确到小数点后30位的圆周率值，且在小数点后每10位处都换行：

pi_digits.txt

```
3.1415926535
8979323846
2643383279
```

要动手尝试后续示例，可在编辑器中输入这些数据行，再将文件保存为pi_digits.txt，也可从本书的配套网站（<https://www.nostarch.com/pythoncrashcourse/>）下载该文件。然后，将该文件保存到本程序所在的目录中。

下面的程序打开并读取这个文件，再将其内容显示到屏幕上：

file_reader.py

```
with open('pi_digits.txt') as file_object:
    contents = file_object.read()
    print(contents)
```

在这个程序中，第1行代码做了大量的工作。我们先来看看函数open()。要以任何方式使用文件——哪怕仅仅是打印其内容，都得先打开文件，这样才能访问它。函数open()接受一个参数：要打开的文件的名称。Python在当前执行的文件所在的目录中查找指定的文件。在这个示例中，当前运行的是file_reader.py，因此Python在file_reader.py所在的目录中查找pi_digits.txt。函数open()返回一个表示文件的对象。在这里，open('pi_digits.txt')返回一个表示文件pi_digits.txt的对象；Python将这个对象存储在我们将在后面使用的变量中。

关键字with在不再需要访问文件后将其关闭。在这个程序中，注意到我们调用了open()，但没有调用close()；你也可以调用open()和close()来打开和关闭文件，但这样做时，如果程序存在bug，导致close()语句未执行，文件将不会关闭。这看似微不足道，但未妥善地关闭文件可能会导致数据丢失或受损。如果在程序中过早地调用close()，你会发现需要使用文件时它已关闭（无法访问），这会导致更多的错误。并非在任何情况下都能轻松确定关闭文件的恰当时机，但通过使用前面所示的结构，可让Python去确定：你只管打开文件，并在需要时使用它，Python自会在合适的时候自动将其关闭。

有了表示pi_digits.txt的文件对象后，我们使用方法read()（前述程序的第2行）读取这个文件的全部内容，并将其作为一个长长的字符串存储在变量contents中。这样，通过打印contents的值，就可将这个文本文件的全部内容显示出来：

```
3.1415926535
8979323846
2643383279
```

相比于原始文件，该输出唯一不同的地方是末尾多了一个空行。为何会多出这个空行呢？因为read()到达文件末尾时返回一个空字符串，而将这个空字符串显示出来时就是一个空行。要删除多出来的空行，可在print语句中使用rstrip()：

```
with open('pi_digits.txt') as file_object:
    contents = file_object.read()
    print(contents.rstrip())
```

本书前面说过，Python方法`rstrip()`删除（剥除）字符串末尾的空白。现在，输出与原始文件的内容完全相同：

```
3.1415926535
8979323846
2643383279
```

10.1.2 文件路径

当你将类似`pi_digits.txt`这样的简单文件名传递给函数`open()`时，Python将在当前执行的文件（即.py程序文件）所在的目录中查找文件。

根据你组织文件的方式，有时可能要打开不在程序文件所属目录中的文件。例如，你可能将程序文件存储在了文件夹`python_work`中，而在文件夹`python_work`中，有一个名为`text_files`的文件夹，用于存储程序文件操作的文本文件。虽然文件夹`text_files`包含在文件夹`python_work`中，但仅向`open()`传递位于该文件夹中的文件的名称也不可行，因为Python只在文件夹`python_work`中查找，而不会在其子文件夹`text_files`中查找。要让Python打开不与程序文件位于同一个目录中的文件，需要提供文件路径，它让Python到系统的特定位置去查找。

由于文件夹`text_files`位于文件夹`python_work`中，因此可使用相对文件路径来打开该文件夹中的文件。相对文件路径让Python到指定的位置去查找，而该位置是相对于当前运行的程序所在目录的。在Linux和OS X中，你可以这样编写代码：

```
with open('text_files/filename.txt') as file_object:
```

这行代码让Python到文件夹`python_work`下的文件夹`text_files`中去查找指定的.txt文件。在Windows系统中，在文件路径中使用反斜杠（\）而不是斜杠（/）：

```
with open('text_files\filename.txt') as file_object:
```

你还可以将文件在计算机中的准确位置告诉Python，这样就不用关心当前运行的程序存储在什么地方了。这称为绝对文件路径。在相对路径行不通时，可使用绝对路径。例如，如果`text_files`并不在文件夹`python_work`中，而在文件夹`other_files`中，则向`open()`传递路径`'text_files/filename.txt'`行不通，因为Python只在文件夹`python_work`中查找该位置。为明确地指出你希望Python到哪里去查找，你需要提供完整的路径。

绝对路径通常比相对路径更长，因此将其存储在一个变量中，再将该变量传递给`open()`会有所帮助。在Linux和OS X中，绝对路径类似于下面这样：

```
file_path = '/home/ehmatthes/other_files/text_files/filename.txt'
with open(file_path) as file_object:
```

而在Windows系统中，它们类似于下面这样：

```
file_path = 'C:\Users\ehmatthes\other_files\text_files\filename.txt'
with open(file_path) as file_object:
```

通过使用绝对路径，可读取系统任何地方的文件。就目前而言，最简单的做法是，要么将数据文件存储在程序文件所在的目录，要么将其存储在程序文件所在目录下的一个文件夹（如text_files）中。

注意 Windows系统有时能够正确地解读文件路径中的斜杠。如果你使用的是Windows系统，且结果不符合预期，请确保在文件路径中使用的是反斜杠。

10.1.3 逐行读取

读取文件时，常常需要检查其中的每一行：你可能要在文件中查找特定的信息，或者要以某种方式修改文件中的文本。例如，你可能要遍历一个包含天气数据的文件，并使用天气描述中包含字样sunny的行。在新闻报道中，你可能会查找包含标签<headline>的行，并按特定的格式设置它。

要以每次一行的方式检查文件，可对文件对象使用for循环：

file_reader.py

```
❶ filename = 'pi_digits.txt'

❷ with open(filename) as file_object:
❸     for line in file_object:
        print(line)
```

在❶处，我们将要读取的文件的名称存储在变量filename中，这是使用文件时一种常见的做法。由于变量filename表示的并非实际文件——它只是一个让Python知道到哪里去查找文件的字符串，因此可轻松地将'pi_digits.txt'替换为你要使用的另一个文件的名称。调用open()后，将一个表示文件及其内容的对象存储到了变量file_object中（见❷）。这里也使用了关键字with，让Python负责妥善地打开和关闭文件。为查看文件的内容，我们通过对文件对象执行循环来遍历文件中的每一行（见❸）。

我们打印每一行时，发现空白行更多了：

```
3.1415926535

8979323846
```

```
2643383279
```

为何会出现这些空白行呢？因为在这个文件中，每行的末尾都有一个看不见的换行符，而print语句也会加上一个换行符，因此每行末尾都有两个换行符：一个来自文件，另一个来自print语句。要消除这些多余的空白行，可在print语句中使用rstrip()：

```
filename = 'pi_digits.txt'

with open(filename) as file_object:
    for line in file_object:
        print(line.rstrip())
```

现在，输出又与文件内容完全相同了：

```
3.1415926535
8979323846
2643383279
```

10.1.4 创建一个包含文件各行内容的列表

使用关键字with时，open()返回的文件对象只在with代码块内可用。如果要在with代码块外访问文件的内容，可在with代码块内将文件的各行存储在一个列表中，并在with代码块外使用该列表：你可以立即处理文件的各个部分，也可推迟到程序后面再处理。

下面的示例在with代码块中将文件pi_digits.txt的各行存储在一个列表中，再在with代码块外打印它们：

```
filename = 'pi_digits.txt'

with open(filename) as file_object:
    ❶ lines = file_object.readlines()

    ❷ for line in lines:
        print(line.rstrip())
```

❶处的方法readlines()从文件中读取每一行，并将其存储在一个列表中；接下来，该列表被存储到变量lines中；在with代码块外，我们依然可以使用这个变量。在❷处，我们使用一个简单的for循环来打印lines中的各行。由于列表lines的每个元素都对应于文件中的一行，因此输出与文件内容完全一致。

10.1.5 使用文件的内容

将文件读取到内存中后，就可以以任何方式使用这些数据了。下面以简单的方式使用圆周率的值。首先，我们将创建一个字符串，它包含文件中存储的所有数字，且没有任何空格：

pi_string.py

```
filename = 'pi_digits.txt'

with open(filename) as file_object:
    lines = file_object.readlines()

❶ pi_string = ''
❷ for line in lines:
    pi_string += line.rstrip()

❸ print(pi_string)
   print(len(pi_string))
```

就像前一个示例一样，我们首先打开文件，并将其中的所有行都存储在一个列表中。在❶处，我们创建了一个变量——pi_string，用于存储圆周率的值。接下来，我们使用一个循环将各行都加入pi_string，并删除每行末尾的换行符（见❷）。在❸处，我们打印这个字符串及其长度：

```
3.1415926535 8979323846 2643383279
36
```

在变量pi_string存储的字符串中，包含原来位于每行左边的空格，为删除这些空格，可使用strip()而不是rstrip()：

```
filename = 'pi_30_digits.txt'

with open(filename) as file_object:
    lines = file_object.readlines()

pi_string = ''
for line in lines:
    pi_string += line.strip()

print(pi_string)
print(len(pi_string))
```

这样，我们就获得了一个这样的字符串：它包含精确到30位小数的圆周率值。这个字符串长32字符，因为它还包含整数部分的3和小数点：

```
3.141592653589793238462643383279
32
```

注意 读取文本文件时，Python将其中的所有文本都解读为字符串。如果你读取的是数字，并将其作为数值使用，就必须使用函数int()将其转换为整数，或使用函数float()将其转换为浮点数。

10.1.6 包含一百万位的大型文件

前面我们分析的都是一个只有三行的文本文件，但这些代码示例也可处理大得多的文件。如果我们有一个文本文件，其中包含精确到小数点后1 000 000位而不是30位的圆周率值，也可创建一个包含所有这些数字的字符串。为此，我们无需对前面的程序做任何修改，只需将这个文件传递给它即可。在这里，我们只打印到小数点后50位，以免终端为显示全部1 000 000位而不断地翻滚：

pi_string.py

```
filename = 'pi_million_digits.txt'

with open(filename) as file_object:
    lines = file_object.readlines()

pi_string = ''
for line in lines:
    pi_string += line.strip()

print(pi_string[:52] + "...")
print(len(pi_string))
```

输出表明，我们创建的字符串确实包含精确到小数点后1 000 000位的圆周率值：

```
3.14159265358979323846264338327950288419716939937510...
1000002
```

对于你可处理的数据量，Python没有任何限制；只要系统的内存足够多，你想处理多少数据都可以。

注意 要运行这个程序（以及后面的众多示例），你需要从<https://www.nostarch.com/pythoncrashcourse/>下载相关的资源。

10.1.7 圆周率值中包含你的生日吗

我一直想知道自己的生日是否包含在圆周率值中。下面来扩展刚才编写的程序，以确定某个人的生日是否包含在圆周率值的前1 000 000位中。为此，可将生日表示为一个由数字组成的字符串，再检查这个字符串是否包含在pi_string中：

```
filename = 'pi_million_digits.txt'

with open(filename) as file_object:
    lines = file_object.readlines()
```

```
pi_string = ''
for line in lines:
    pi_string += line.rstrip()

❶ birthday = input("Enter your birthday, in the form mmddyy: ")
❷ if birthday in pi_string:
    print("Your birthday appears in the first million digits of pi!")
else:
    print("Your birthday does not appear in the first million digits of pi.")
```

在❶处，我们提示用户输入其生日，在接下来的❷处，我们检查这个字符串是否包含在pi_string中。运行一下这个程序：

```
Enter your birthdate, in the form mmddyy: 120372
Your birthday appears in the first million digits of pi!
```

我的生日确实出现在了圆周率值中！读取文件的内容后，就可以以你能想到的任何方式对其进行分析。

动手试一试

10-1 Python 学习笔记：在文本编辑器中新建一个文件，写几句话来总结一下你至此学到的 Python 知识，其中每一行都以“In Python you can”打头。将这个文件命名为 learning_python.txt，并将其存储到为完成本章练习而编写的程序所在的目录中。编写一个程序，它读取这个文件，并将你所写的内容打印三次：第一次打印时读取整个文件；第二次打印时遍历文件对象；第三次打印时将各行存储在一个列表中，再在 with 代码块外打印它们。

10-2 C 语言学习笔记：可使用方法 replace()将字符串中的特定单词都替换为另一个单词。下面是一个简单的示例，演示了如何将句子中的'dog'替换为'cat'：

```
>>> message = "I really like dogs."
>>> message.replace('dog', 'cat')
'I really like cats.'
```

读取你刚创建的文件 learning_python.txt 中的每一行，将其中的 Python 都替换为另一门语言的名称，如 C。将修改后的各行都打印到屏幕上。

10

10.2 写入文件

保存数据的最简单的方式之一是将其写入到文件中。通过将输出写入文件，即便关闭包含程序输出的终端窗口，这些输出也依然存在：你可以在程序结束运行后查看这些输出，可与别人分享输出文件，还可编写程序来将这些输出读取到内存中并进行处理。

10.2.1 写入空文件

要将文本写入文件，你在调用`open()`时需要提供另一个实参，告诉Python你要写入打开的文件。为明白其中的工作原理，我们来将一条简单的消息存储到文件中，而不是将其打印到屏幕上：

```
write_message.py
```

```
filename = 'programming.txt'
```

```
❶ with open(filename, 'w') as file_object:
    ❷     file_object.write("I love programming.")
```

在这个示例中，调用`open()`时提供了两个实参（见❶）。第一个实参也是要打开的文件的名称；第二个实参（'w'）告诉Python，我们要以写入模式打开这个文件。打开文件时，可指定读取模式（'r'）、写入模式（'w'）、附加模式（'a'）或让你能够读取和写入文件的模式（'r+'）。如果你省略了模式实参，Python将以默认的只读模式打开文件。

如果你要写入的文件不存在，函数`open()`将自动创建它。然而，以写入（'w'）模式打开文件时千万要小心，因为如果指定的文件已经存在，Python将在返回文件对象前清空该文件。

在❷处，我们使用文件对象的方法`write()`将一个字符串写入文件。这个程序没有终端输出，但如果你打开文件`programming.txt`，将看到其中包含如下一行内容：

```
programming.txt
```

```
I love programming.
```

相比于你的计算机中的其他文件，这个文件没有什么不同。你可以打开它、在其中输入新文本、复制其内容、将内容粘贴到其中等。

注意 Python只能将字符串写入文本文件。要将数值数据存储到文本文件中，必须先使用函数`str()`将其转换为字符串格式。

10.2.2 写入多行

函数`write()`不会在你写入的文本末尾添加换行符，因此如果你写入多行时没有指定换行符，文件看起来可能不是你希望的那样：

```
filename = 'programming.txt'
```

```
with open(filename, 'w') as file_object:
    file_object.write("I love programming.")
    file_object.write("I love creating new games.")
```

如果你打开`programming.txt`，将发现两行内容挤在一起：

```
I love programming.I love creating new games.
```

要让每个字符串都单独占一行，需要在`write()`语句中包含换行符：

```
filename = 'programming.txt'

with open(filename, 'w') as file_object:
    file_object.write("I love programming.\n")
    file_object.write("I love creating new games.\n")
```

现在，输出出现在不同行中：

```
I love programming.
I love creating new games.
```

像显示到终端的输出一样，还可以使用空格、制表符和空行来设置这些输出的格式。

10.2.3 附加到文件

如果你要给文件添加内容，而不是覆盖原有的内容，可以附加模式打开文件。你以附加模式打开文件时，Python不会在返回文件对象前清空文件，而你写入到文件的行都将添加到文件末尾。如果指定的文件不存在，Python将为你创建一个空文件。

下面来修改`write_message.py`，在既有文件`programming.txt`中再添加一些你酷爱编程的原因：

`write_message.py`

```
filename = 'programming.txt'

❶ with open(filename, 'a') as file_object:
❷     file_object.write("I also love finding meaning in large datasets.\n")
    file_object.write("I love creating apps that can run in a browser.\n")
```

在❶处，我们打开文件时指定了实参'a'，以便将内容附加到文件末尾，而不是覆盖文件原来的内容。在❷处，我们又写入了两行，它们被添加到文件`programming.txt`末尾：

`programming.txt`

```
I love programming.
I love creating new games.
I also love finding meaning in large datasets.
I love creating apps that can run in a browser.
```

最终的结果是，文件原来的内容还在，它们后面是我们刚添加的内容。